Credit Lecture 1: Introduction and Data Example

Deep Learning for Actuarial Modelling: a credit risk companion

AUTHOR

Mario Ervedosa, adapting Scognamiglio, Maggi and Wüthrich

PUBLISHED

August 31, 2026

ABSTRACT

We introduce a consumer credit probability of default (PD) problem, which will serve as the running example for a credit risk reading of the summer school lectures. The storyline and notation follow Lecture 1 of Deep Learning for Actuarial Modeling; the French motor insurance portfolio is replaced by the Bondora peer-to-peer loan book, and claim frequency modelling becomes default modelling. The analysis in this first lecture is mostly empirical.

1 Credit risk modelling: a PD problem

We introduce a consumer credit modelling problem. This problem is based on **probabilities of default (PDs)**, the credit analogue of the claims frequencies that drive the insurance pricing problem of the original lecture. The analysis in this first lecture will mostly be empirical. It considers the loan book of Bondora, an Estonian peer-to-peer lending platform, whose full origination history is public. The book covers 179,235 loans originated between February 2009 and July 2021 in four countries, with Estonia, Finland and Spain carrying nearly all of the volume.

Generally, raw data needs **data cleaning**, e.g., correcting for data errors, inputting missing values, merging partial information, etc. The Bondora extract is raw in exactly this sense. Missing values arrive as empty strings, decimals are written like `.6800`, and 53 loans carry a borrower age below 18, Bondora's contractual minimum. The cleaning applied here lives in `scripts/convert_credit_data.py`, which also selects the columns a lender would hold **at application time** and derives the default outcome defined below. Post-origination columns (payment histories, recoveries, balances and the platform's own loss estimates) are excluded from the modelling table by construction, because they leak the outcome into the covariates.

1.1 Load and illustrate the Bondora data

```
import numpy as np
import polars as pl
import matplotlib.pyplot as plt

dat = pl.read_parquet("../data/bondora_pd.parquet")
dat.select(
    ["LoanNumber", "LoanDate", "Age", "Country", "Amount", "Interest",
     "LoanDuration", "Education", "IncomeTotal", "DebtToIncome", "Rating",
     "default_12m"]
).head(6)
```

shape: (6, 12)

LoanNumber	LoanDate	Age	Country	Amount	Interest	LoanDuration	Education	IncomeTotal	DebtToIncome	Rating	defau
i64	date	i64	str	f64	f64	i64	i64	f64	f64	str	
483449	2016-03-23	53	"EE"	2125.0	20.97	60	4	354.0	26.29	"C"	
378148	2015-06-25	50	"EE"	3000.0	17.12	60	5	900.0	30.58	"B"	
451831	2016-01-19	44	"EE"	9100.0	13.67	60	4	1200.0	26.71	"A"	
349381	2015-03-27	42	"ES"	1500.0	40.4	60	2	863.0	7.36	"F"	
443082	2015-12-22	34	"ES"	1090.0	68.39	48	4	697.0	36.04	"HR"	
430905	2015-11-19	31	"ES"	775.0	73.73	60	4	970.0	47.96	"HR"	

The modelling table contains 49 columns. The variables used in this lecture are the following:

- `LoanNumber` is a unique loan identifier
- `LoanDate` is the origination date, which defines the vintage
- `Age` is the age of the borrower at application
- `Country` is the borrower's country of residence (categorical)
- `Amount` is the funded loan amount in EUR
- `Interest` is the contractual interest rate in per cent per annum
- `LoanDuration` is the scheduled term in months
- `Education` is the borrower's education level (ordinal categorical)
- `IncomeTotal` is the borrower's total declared monthly income in EUR
- `DebtToIncome` is the declared debt-to-income ratio
- `Rating` is Bondora's own credit grade assigned at listing (categorical)
- `default_12m` indicates default within twelve months of origination

The remaining columns describe employment, the income decomposition, liabilities and the borrower's history on the platform. They enter the story in the later lectures. For simplicity, we focus on the binary default indicator `default_12m`; the raw extract would also support loss given default and exposure at default modelling through its recovery columns.

2 The outcome: default, windows and cohorts

2.1 The definition of default

In insurance, a claim is a claim. In credit, the event we model is a matter of definition, and the definition is regulated. Article 178 of the Capital Requirements Regulation sets the objective backstop for regulatory capital models. An obligor is in default once they are more than 90 days past due on a material credit obligation, and earlier if the lender judges them unlikely to pay. The PRA's supervisory statement SS3/24 adds expectations on how the materiality threshold must be applied, so that threshold choices do not artificially suppress observed default rates in the calibration data.

Bondora records its own event instead. The field `DefaultDate` marks the date a loan entered the platform's defaulted state and the collection process started. We therefore work with a **platform definition of default** and note that a bank building a regulatory model would rebuild the flag from payment arrears under Article 178. For a first lecture the distinction changes nothing methodologically; every model below applies verbatim to a 90-days-past-due flag.

THE DEFAULT DEFINITION AS A DESIGN CHOICE

Article 178 fixes a regulatory floor and the platform fixes its own flag, but a modeller building a behavioural or IFRS 9 model chooses. Botha, Oberholzer, Larney and de Jongh (2023) formalise that choice with three parameters. Let $g_0(t)$ measure a loan's delinquency at month t, counted in payments in arrears. A preliminary event is declared at t when

$$\mathcal{G}(d, s, t) = \left[\sum_{v=t-(s-1)}^{t} [g_0(v) \geq d] = s \right],$$

writing $[\cdot]$ for Iverson brackets, so the event fires only where delinquency holds at or above the threshold d for s consecutive months. The modelling outcome at t then reads that status k months ahead, $y_t = \mathcal{G}(d, s, t + k)$.

Three dials, therefore: the delinquency threshold d, the stickiness s, and the outcome period k. Their study sweeps $d \in \{1, 2\}$, $s \in \{1, 2, 3\}$ and $k \in \{3, 6, 9, 12\}$, and treats three payments in arrears, $g_0(t) \geq 3$, as default itself. Read against Bondora:

Dial	Meaning	Bondora's choice
d	delinquency threshold	roughly two payments, since the platform flags at 60 days past due
s	consecutive months tested	$s = 1$, since one `DefaultDate` fires the flag with no persistence test
k	outcome period	twelve months here, and free to vary from the next section on

The severity dial is where the two traditions meet. Three payments in arrears sits at roughly ninety days, which is where Article 178 sits, so the regulatory definition and the academic one agree on d and differ over whether the modeller may touch s and k at all.

2.2 Deriving the twelve-month flag

HOW DEFAULT_12M IS BUILT

Three steps, all of them in `scripts/convert_credit_data.py`.

1. **Fix the horizon.** The newest origination in the extract is 19 July 2021, so keep only loans originated on or before 19 July 2020. Every retained loan then has a full twelve months of observation behind it.
2. **Flag the event.** Set `default_12m` to 1 where `DefaultDate` falls within 365 days of `LoanDate`.
3. **Fill the rest.** A loan carrying no `DefaultDate` never entered the platform's defaulted state, so it scores 0.

Three loans make the mechanics concrete.

Loan	LoanDate	DefaultDate	Retained	default_12m
A	2019-03-14	2019-11-02	yes	1
B	2019-06-08	2021-02-17	yes	0
C	2020-11-30	2021-06-05	no	not scored

Loan B is the one to sit with. It defaulted, and the flag still records a zero, because the default arrived twenty months after origination while the window closed at twelve. Loan C defaulted inside its first twelve months, but only eight of those months had elapsed by the extract date, so the loan is dropped rather than scored. Both outcomes are deliberate. The next two sections are about what they cost.

2.3 Censoring

A second issue arrives with the outcome rather than with the covariates, and general insurance offers no counterpart to it in this form. The reason is worth setting out, because it points at where credit's actuarial cousins actually live.

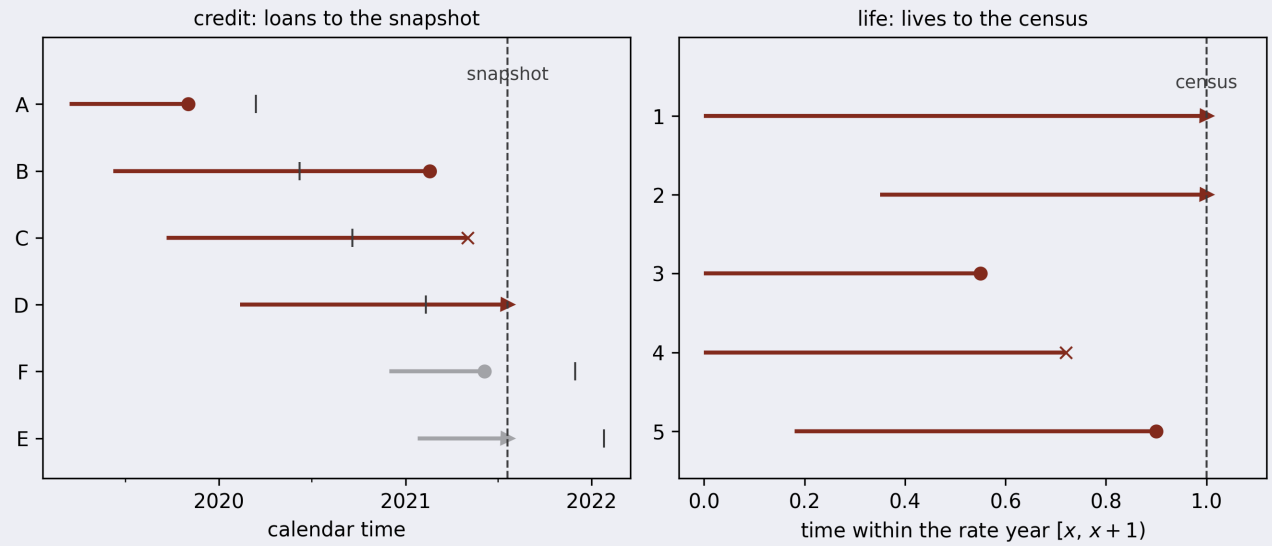
CENSORING, AND WHY CREDIT BORROWS IT FROM LIFE RATHER THAN FROM GENERAL INSURANCE

At the extract date 57,774 loans were still current. A loan originated three months earlier has had no time to default, and scoring it a non-default would bias every estimate downwards. The event time exists; we have simply not waited long enough to see it. That is **right-censoring**.

Life insurance meets the identical problem whenever a mortality rate is calibrated. A life observed from entry to a census date, still alive at the census, tells you it survived that far and nothing more. The denominator of q_x therefore counts time actually at risk rather than lives, and the central exposed to risk is precisely that denominator. Lives entering part-way through the rate year contribute part-years, and lives withdrawing contribute only up to withdrawal.

General insurance escapes the problem. The exposure v_i is the earned policy period, which is measured and complete once the period has run, so no latent event date waits beyond the edge of the data. Incompleteness in general insurance sits instead in claim reporting and development, i.e. IBNR and IBNER, and that is a question about the size and timing of claims arising on a known exposure. Life and credit share right-censoring because both wait on an absorbing event whose date may fall outside the observation window; general insurance counts events inside a window it has already observed in full.

```
import datetime as dt
import matplotlib.dates as mdates

OX, GREY, RULE = "#8C2F1E", "#B0B0B0", "#444444"
tau_d = dt.date(2021, 7, 20)

# label, origination, exit date, exit kind, retained by the k = 12 filter
loans = [
    ("A", dt.date(2019, 3, 14), dt.date(2019, 11, 2), "default", True),
    ("B", dt.date(2019, 6, 8), dt.date(2021, 2, 17), "default", True),
    ("C", dt.date(2019, 9, 20), dt.date(2021, 5, 3), "repaid", True),
    ("D", dt.date(2020, 2, 11), None, "open", True),
    ("F", dt.date(2020, 11, 30), dt.date(2021, 6, 5), "default", False),
    ("E", dt.date(2021, 1, 25), None, "open", False),
]

fig, axs = plt.subplots(1, 2)

ax = axs[0]
for j, (lab, g, out, kind, kept) in enumerate(loans):
    y = len(loans) - j
    col = OX if kept else GREY
    end = out or tau_d
    ax.plot([g, end], [y, y], color=col, lw=2, solid_capstyle="butt")
    ax.plot([g + dt.timedelta(days=365)], [y], marker="|", ms=9, color=RULE)
    marker = {"default": "o", "repaid": "x", "open": ">"}[kind]
    ax.plot([end], [y], marker, color=col, ms=6)
ax.axvline(tau_d, color=RULE, ls="--", lw=1)
ax.annotate("snapshot", xy=(tau_d, len(loans) + 0.30), ha="center", va="bottom",
            fontsize=9, color=RULE, annotation_clip=False)
ax.set_yticks(range(1, len(loans) + 1))
ax.set_yticklabels([r[0] for r in reversed(loans)])
ax.set_ylim(0.4, len(loans) + 1.0)
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
ax.xaxis.set_minor_locator(mdates.MonthLocator(bymonth=[7]))
ax.set_xlabel("calendar time")
ax.set_title("credit: loans to the snapshot", fontsize=10)

ax = axs[1]
# label, entry, exit (fraction of the rate year), exit kind
lives = [("1", 0.00, 1.00, "open"), ("2", 0.35, 1.00, "open"),
         ("3", 0.00, 0.55, "death"), ("4", 0.00, 0.72, "withdrawn"),
         ("5", 0.18, 0.90, "death")]
for j, (lab, a, b, kind) in enumerate(lives):
    y = len(lives) - j
    ax.plot([a, b], [y, y], color=OX, lw=2, solid_capstyle="butt")
    marker = {"death": "o", "withdrawn": "x", "open": ">"}[kind]
    ax.plot([b], [y], marker, color=OX, ms=6)
ax.axvline(1.0, color=RULE, ls="--", lw=1)
ax.annotate("census", xy=(1.0, len(lives) + 0.30), ha="center", va="bottom",
            fontsize=9, color=RULE, annotation_clip=False)
ax.set_yticks(range(1, len(lives) + 1))
ax.set_yticklabels([r[0] for r in reversed(lives)])
ax.set_ylim(0.4, len(lives) + 1.0)
ax.set_xlim(-0.05, 1.12)
ax.set_xlabel("time within the rate year $[x,\\,x+1)$")
ax.set_title("life: lives to the census", fontsize=10)

plt.tight_layout(); plt.show()
```



Right-censoring in two settings, drawn on illustrative cases rather than portfolio data. (lhs): six loans between origination and the snapshot τ = 20 July 2021, dashed. The grey tick on each bar marks $g_i + 12$ months, the close of the outcome window; grey bars are loans the fixed-horizon filter drops for insufficient seasoning. (rhs): five lives inside one rate year $[x, x+1)$, on which the central exposed to risk for q_x is computed. Circles mark the event of interest (default, death), crosses a competing exit (repayment, withdrawal) and arrowheads an observation still open at the boundary.

2.4 A general outcome window

We resolve the censoring with a **fixed-horizon** outcome, the standard device of application scorecards. Build it from the monthly default flag directly. Let $y_{i,t} \in \{0, 1\}$ record whether loan i is in default at t months on book, i.e. at calendar month $g_i + t$, under the Article 178 definition set out above. The time to default is then the first month that flag fires,

$$T_i = \min \{t \geq 1 : y_{i,t} = 1\},$$

with $T_i = \infty$ for a loan that never defaults. Let further

$$A_i = \tau - g_i$$

be the months of observation available on loan i at the snapshot date τ, where g_i is its origination month. For an outcome window of k months,

$$D_i^{(k)} = \mathbf{1}\{T_i \leq k\} = \max(y_{i,1}, \ldots, y_{i,k}), \qquad \mathcal{W}_k = \{i : A_i \geq k\}, \qquad n(k) = \#\mathcal{W}_k,$$

where $\#\mathcal{W}_k$ denotes the number of elements of the set, its cardinality. The second equality for $D_i^{(k)}$ is worth pausing on, because it is Botha's **worst-ever** aggregation appearing here already: a loan that breaches and then cures still scores a one, since the maximum over the window remembers the breach. The same device returns in the next section as the ingredient of a point-in-time default rate.

The retention rule $\mathcal{W}_k$ keeps only loans whose **w**indow had closed by the snapshot, so every retained loan carries a complete observation. It takes a plain letter deliberately: $\mathcal{L}$ is the loss function throughout the summer school lectures, and reusing it here would collide. Setting $k = 12$ reproduces `default_12m` exactly, at $n(12) = 148{,}733$ loans, and that is what the rest of this lecture uses. Every formula from here on carries k, so nothing needs rewriting when the window moves. Note also that k is the outcome period of the callout above, i.e. the third of Botha's three dials.

The window is not a free parameter in practice, and the cost of moving it shows up immediately.

```
raw = pl.read_parquet(
    "../data/bondora_raw.parquet",
    columns=["ReportAsOfEOD", "LoanDate", "DefaultDate"],
)
MONTH = 30.4375  # mean Gregorian month in days
tau = raw["ReportAsOfEOD"][0]
raw = raw.with_columns(
    A=(pl.lit(tau) - pl.col("LoanDate")).dt.total_days() / MONTH,
    T=(pl.col("DefaultDate") - pl.col("LoanDate")).dt.total_days() / MONTH,
)
pl.DataFrame(
    [
        {
            "k (months)": k,
            "n(k)": kept.height,
            "default rate": (kept["T"] <= k).fill_null(False).mean(),
        }
        for k in (3, 6, 12, 24, 36, 48, 60)
        for kept in [raw.filter(pl.col("A") >= k)]
    ]
)
```

shape: (7, 3)

k (months)	n(k)	default rate
i64	i64	f64
3	166771	0.001553
6	158744	0.128332
12	148733	0.289492
24	98027	0.454609
36	60755	0.5126
48	39081	0.534633
60	25971	0.521351

Three features of that table matter, and they arise from two different mechanisms.

At $k = 3$ the rate is 0.16 per cent, which measures nothing at all. Bondora flags default at 60 days past due, and a first instalment cannot be 60 days late until roughly ninety days after origination, so the window closes before the definition can fire. The outcome window and the default definition interact, which is why the two cannot be chosen independently.

Between $k = 12$ and $k = 24$ the rate climbs from 28.95 to 45.46 per cent, then flattens. That part is genuine saturation: the observed time to default has its ninetieth percentile at 721 days, so by two years most defaults that will happen have happened.

At $k = 60$ the rate falls back to 52.14 per cent from 53.46. That part is composition rather than risk. The filter $A_i \geq 60$ retains only loans originated before mid-2016, so the sample itself changes as the window lengthens, and the later high-risk vintages drop out of it. Botha, Oberholzer, Larney and de Jongh (2023) describe both failure modes from the literature they review: too short a window gives volatile results driven by risk immaturity, and too long a window drifts out of step with market conditions and with the portfolio's own risk composition. The section on cohorts below is where that second problem gets its name.

2.5 Cohorts, vintages and calendar time

Three time axes run through any loan book, and keeping them apart is what lets the actuarial machinery transfer across intact.

Symbol	Meaning	Life insurance twin
g_i	vintage, the origination month of loan i	year of birth
t	months on book, the attained age of the loan	attained age x
u	calendar month, $u = g + t$	calendar year of observation
k	the horizon being asked about, in months	the duration t in ${}_tp_x$
τ	the snapshot date	census date
$A_i = \tau - g_i$	months of observation available	exposure to the census

Note which letter plays which role, because the tempting mapping is the wrong one. A loan's own clock is months on book, so t is the loan's **attained age**, and the actuarial twin of t is x rather than the duration. Vintage g_i is then the loan's date of birth, which is what makes $u = g + t$ the same identity a demographer draws. Consequently the survival probability transliterates as

$$ {}_kp_{i,t} = \mathbb{P}\left(\text{loan } i,\ \text{now } t \text{ months on book, survives a further } k \text{ months}\right), $$

which has exactly the shape of ${}_tp_x$ with (k, t) in the roles of (t, x). Note that t is therefore unbounded by k: a loan can be forty months on book and still be asked about over the next twelve.

THE SAME STATEMENT IN LIFE-TABLE NOTATION

Write the same thing as a life table and the correspondence becomes mechanical. Fix a vintage g and follow its loans forward. Let l_t count the loans of that vintage still in force and performing at t months on book, l_0 being the number originated, and let d_t^{def} and d_t^{set} count those leaving between t and $t+1$ by default and by settlement. The cohort runs down as

$$l_{t+1} = l_t - d_t^{\text{def}} - d_t^{\text{set}},$$

so a ratio of two of those counts is a conditional survival probability,

$$\frac{l_{k+t}}{l_t} = \mathbb{P}\left(\text{in force and performing at } k+t \,\middle|\, \text{in force and performing at } t\right).$$

Where default is the only way out, i.e. $d_t^{\text{set}} \equiv 0$, that ratio is exactly the object the paragraph above named, now written through the time to default T_i of section 2.4:

$${}_kp_t = \frac{l_{k+t}}{l_t} = \mathbb{P}\left(T_i > k+t \,|\, T_i > t\right), \qquad {}_kq_t = 1 - {}_kp_t = \frac{l_t - l_{k+t}}{l_t} = \frac{1}{l_t}\sum_{s=t}^{k+t-1} d_s^{\text{def}}.$$

The subscript i has gone because a life table is a statement about a cohort; ${}_kp_{i,t}$ above is the loan-level version of the same quantity. The conditioning event $\{T_i > t\}$ is what "loans that have never defaulted" amounts to formally. ${}_kp_t$ is asked of the survivors alone, exactly as ${}_tp_x$ is asked of the lives who reached age x, and a defaulted loan has left the table for good, since default is absorbing.

Setting $t = 0$ recovers this lecture's own outcome. $T_i \geq 1$ by construction, so $\{T_i > 0\}$ is the whole cohort and

$${}_kq_0 = \mathbb{P}\left(T_i \leq k\right) = \mathbb{E}\left[D_i^{(k)}\right],$$

which at $k = 12$ is the portfolio default rate the previous section tabulated. The fixed-horizon flag is the first row of this table and nothing more, and the rows beneath it are the term structure lecture S1 goes after. Read ${}_kq_0$ there as the pure-default probability, since $D_i^{(k)}$ is built from the monthly flags $y_{i,t}$ rather than from the counts l_t; the ratio $(l_0 - l_k)/l_0$ recovers the same number only under the assumption the next paragraph examines.

Settlement, however, is common. The survival table built in `scripts/convert_credit_data.py` counts 42,144 loans settling against 71,416 defaulting, so l_{k+t}/l_t as measured on Bondora is a **multiple-decrement** probability, the credit twin of a life table carrying withdrawals. Reading it as a default probability means treating settlement as censoring, which assumes the two decrements independent given the covariates. Lecture S2 takes that assumption apart and supplies the Fine and Gray correction for where it fails.

Borrower age deserves a warning rather than a row in that table. It is written x_i here and it is a genuine and useful covariate, entering the models later in this lecture as an age class. Nevertheless it is **not** the credit twin of the actuarial x, and treating it as one is the most common false friend in this translation. Two borrowers of different ages whose loans are both twelve months old sit at the same point on the loan's survival curve; their ages shift the level of the curve without indexing it.

The three axes are then bound by an identity. For loan i observed at t months on book, the calendar month is

$$u = g_i + t,$$

and the subscript is dropped only when the axes are discussed as axes rather than as quantities attached to a particular loan.

Only two of the three are ever free. Demographers know this as the age-period-cohort identification problem, and with t read as the loan's attained age the correspondence is exact: **age** is months on book t, **period** is calendar month u, and **cohort** is vintage g. That is why a mortality model indexed by all three needs a constraint imposed by hand instead of estimated, and credit inherits the problem whole. Breeden's age-period-cohort decomposition of loan-level performance is the same identity in credit clothing, splitting observed performance into a lifecycle in t, an environment in u and a credit-quality effect in g.

One property of this extract now does more damage than it appears to. `ReportAsOfEOD` takes a single value across all 179,235 rows, 20 July 2021. The snapshot τ is therefore a constant rather than a per-row field, and the extract holds one cross-section where a panel would hold many. What that costs is taken up once the PD itself has been defined, in the callout closing the next section.

Five real loans put every symbol of this section on a row. One per censoring pattern, picked as the lowest `LoanNumber` matching each, so the selection is reproducible rather than sampled. That rule reaches for the oldest loans on the platform, which is why three of the five carry the March 2009 vintage.

```
ex = pl.read_parquet(
    "../data/bondora_raw.parquet",
    columns=["LoanNumber", "LoanDate", "DefaultDate", "Status"],
).with_columns(
    g=pl.col("LoanDate").dt.strftime("%Y-%m"),
    A=(pl.lit(tau) - pl.col("LoanDate")).dt.total_days() / MONTH,
    T=(pl.col("DefaultDate") - pl.col("LoanDate")).dt.total_days() / MONTH,
).with_columns(retained=pl.col("A") >= 12)

patterns = {
    "default inside the window": pl.col("retained") & (pl.col("T") <= 12),
    "default after the window": pl.col("retained") & (pl.col("T") > 12),
    "settled, never defaulted": pl.col("retained") & pl.col("T").is_null()
                                & (pl.col("Status") == "Repaid"),
    "still open at the snapshot": pl.col("retained") & pl.col("T").is_null()
                                  & (pl.col("Status") == "Current"),
    "dropped, not yet seasoned": ~pl.col("retained") & (pl.col("T") <= 12),
}
pl.concat(
    ex.filter(cond).sort("LoanNumber").head(1).select(
        pl.lit(name).alias("pattern"),
        "LoanNumber",
        pl.col("g").alias("g_i"),
        pl.col("A").round(2).alias("A_i"),
        pl.col("T").round(2).alias("T_i"),
        pl.when("retained")
          .then((pl.col("T") <= 12).fill_null(False))
          .cast(pl.Int8).alias("D_i(12)"),
        pl.col("retained").alias("in W_12"),
    )
    for name, cond in patterns.items()
)
```

shape: (5, 7)

pattern	LoanNumber	g_i	A_i	T_i	D_i(12)	in W_12
str	i64	str	f64	f64	i8	bool
"default inside the window"	163	"2009-03"	148.17	3.06	1	true
"default after the window"	45	"2009-03"	148.44	13.9	0	true
"settled, never defaulted"	37	"2009-03"	148.44	null	0	true
"still open at the snapshot"	13581	"2012-11"	104.44	null	0	true
"dropped, not yet seasoned"	2063289	"2020-07"	11.99	3.71	null	false

Two columns of this section are deliberately absent. The snapshot τ would print 20 July 2021 on every row, being the constant just discussed, and the calendar month $u = g_i + t$ is an identity rather than a measurement, recoverable from the vintage and whichever t the reader cares to ask about. Months on book at the snapshot is A_i itself, so that column is already there under its other name.

Read the last two rows together, because they are the ones that cost something. Loan 13581 was originated in November 2012 and is still current at the snapshot, so it contributes a genuine zero to `default_12m`: its window closed years ago. Loan 2063289 defaulted after 3.71 months, well inside twelve, and is dropped all the same, since only $A_i = 11.99$ months of observation had accumulated by

the snapshot. Note what that figure exposes. The loan was originated exactly 365 days before the snapshot, and dividing days by the mean Gregorian month of 30.4375 puts it a hundredth of a month short of the boundary. The filter $A_i \geq 12$ therefore draws the same line as the 365-day rule in `scripts/convert_credit_data.py`, which is why $n(12)$ reproduces `default_12m` exactly.

2.6 Default modelling and PDs

Fix the window at $k = 12$ for the rest of this lecture. The portfolio is then the retained set $\mathcal{W}_{12}$ of the section above, whose size we named there: $n := n(12) = 148{,}733$ loans, which we label $i \in \{1, \dots, n\}$. Each loan is characterised by covariates $\boldsymbol{X}_i$ describing the borrower and the contract at application time, such as the age of the borrower or the loan amount.

The original lecture models claim counts $N_i \geq 0$ observed over a time exposure $v_i \in (0, 1]$, and studies the claims frequencies $Y_i = N_i / v_i$. That notation carries over to credit almost unchanged. Within the observation window, loan i produces a number of default events N_i, and the window itself plays the role of the exposure v_i. Default is an absorbing state, so $N_i \in \{0, 1\}$ rather than $N_i \in \{0, 1, 2, \dots\}$.

What the exposure should be is then a real choice, and the two answers lead to different models. Measuring exposure in windows gives every retained loan the same one, $v_i \equiv 1$ window of k months, and the frequency collapses to the indicator itself,

$$Y_i^{(k)} = N_i / v_i = D_i^{(k)} \in \{0, 1\}.$$

Measuring it in months at risk instead gives each loan the time it actually spent exposed,

$$v_i = \min(T_i,\, k) \text{ months} \qquad \Longrightarrow \qquad Y_i^{(k)} = \frac{N_i}{\min(T_i,\, k)},$$

which is the honest claims-frequency analogue, with time to default in the role of exposure. Here the frequency does not collapse. A loan defaulting in month two and a loan defaulting in month eleven both score $D_i^{(k)} = 1$, yet they carry intensities differing by a factor of five, and the second convention keeps that difference while the first discards it.

The first convention is how application scorecards are built, and it is what this lecture uses. The second is a discrete hazard, and it is the door into survival analysis: survival and hazard-rate models treat default as an event arriving at some intensity over time on book, and the IFRS 9 lifetime PD term structure is built exactly that way. Lecture S1 walks through that door, starting from the exposure $v_i = \min(T_i, k)$ defined here.

The general task is to predict (forecast) the default behaviour as accurately as possible, taking into account all available borrower information $\boldsymbol{X}_i$. (Best) predictors are typically computed by conditional means, and the quantity admits two equivalent statements. Asked of the time to default, what is wanted is the probability that the default arrives inside the window,

$$\mathrm{PD}_k(\boldsymbol{X}_i) := \mathbb{P}\left(T_i \leq k \,|\, \boldsymbol{X}_i\right).$$

That is the survival-analysis statement: a distribution function of T_i evaluated at k, and the covariate-conditional version of the ${}_kq_{i,0}$ that the life-table callout above obtains by reading ${}_kp_{i,t}$ at $t = 0$. A term structure of PDs is the same object read at several k at once, and lectures S1 and S2 work at this level throughout.

The regression machinery wants the indicator instead, and the bridge between the two is the definition $D_i^{(k)} = \mathbf{1}\{T_i \leq k\}$, since the expectation of an indicator is the probability of the event it indicates. Hence

$$\mu_k(\boldsymbol{X}_i) := \mathbb{E}\left[D_i^{(k)} \,\middle|\, \boldsymbol{X}_i\right] = \mathbb{P}\left(D_i^{(k)} = 1 \,\middle|\, \boldsymbol{X}_i\right) = \mathbb{P}\left(T_i \leq k \,|\, \boldsymbol{X}_i\right) = \mathrm{PD}_k(\boldsymbol{X}_i).$$

For a binary response, therefore, the conditional mean **is** the probability of default, and it carries the window it was measured over wherever it goes, which is why k sits on μ_k throughout. It is the middle form, conditioning on covariates and averaging an indicator, that the point-in-time callout below extends by conditioning on the calendar month u as well. The pricing functional of the insurance lecture becomes the PD functional

$$\boldsymbol{X} \mapsto \mu_k(\boldsymbol{X}) = \mathrm{PD}_k(\boldsymbol{X}),$$

and everything the lectures develop for estimating regression functions applies to it. The distributional consequence is taken up in the next lecture. Claim counts led to the Poisson member of the exponential dispersion family; a binary indicator leads to the Bernoulli member, whose canonical link is the logit. Deviance losses, maximum likelihood estimation and the balance property all carry over.

Three properties fix what the function $\boldsymbol{X} \mapsto \mathbb{E}[Y \mid \boldsymbol{X}]$ actually is, writing Y for the response and taking $\boldsymbol{X}$ discrete, as the covariates of a scorecard mostly are. Note first that it is a function of $\boldsymbol{X}$ rather than a number, so it is itself a random variable once $\boldsymbol{X}$ is.

First, it averages the response over the conditional distribution,

$$\mu(\boldsymbol{x}) = \mathbb{E}\left[Y \mid \boldsymbol{X} = \boldsymbol{x}\right] = \sum_{y} y\, \mathbb{P}\left(Y = y \mid \boldsymbol{X} = \boldsymbol{x}\right), \qquad \text{or} \qquad \int y\, f_{Y \mid \boldsymbol{X}}(y \mid \boldsymbol{x})\, \mathrm{d}y \quad \text{for continuous } Y.$$

For $Y = D^{(k)} \in \{0, 1\}$ that sum has two terms and the $y = 0$ term vanishes, leaving $\mu_k(\boldsymbol{x}) = \mathbb{P}(D^{(k)} = 1 \mid \boldsymbol{X} = \boldsymbol{x})$. The conditional mean and the conditional probability are then the same number, which is what makes PD_k a regression problem rather than something exotic.

Second, among all functions of the covariates it is the best square-error predictor,

$$\mu = \arg\min_{f} \mathbb{E}\left[\left(Y - f(\boldsymbol{X})\right)^2\right],$$

the minimum running over measurable f with a finite second moment. That is what "(best) predictor" means above. Beyond squared error the property survives intact, since the conditional mean minimises expected loss for every Bregman divergence, the Poisson and Bernoulli deviances of the next lecture included. Changing the loss to suit the distribution therefore leaves μ_k as the target.

Third, it averages back to the unconditional mean,

$$\mathbb{E}\left[\mathbb{E}\left[Y \mid \boldsymbol{X}\right]\right] = \mathbb{E}[Y].$$

The tower property, and in this setting the **balance property** just mentioned. A model estimating μ_k ought to reproduce the portfolio default rate once its predictions are averaged, and lecture 7 is about what breaks that and how to restore it.

The point-in-time question the previous section raised can now be stated in the language of PD_k, which is why it was held back to here.

POINT-IN-TIME AND THROUGH-THE-CYCLE PDS

A **point-in-time (PiT)** PD conditions on where the economy is and moves with the cycle, and IFRS 9 requires one, because an expected credit loss is a forecast made from today's conditions. A **through-the-cycle (TTC)** PD averages the cycle away instead, which is what an IRB long-run average targets. Both need the outcome generalised first, since a PD asked of a loan already t months on book wants a window opening at t. For a loan performing at t,

$$D_{i,t}^{(k)} = \max\left(y_{i,t+1}, \dots, y_{i,t+k}\right) \in \{0,1\}, \qquad D_{i,0}^{(k)} = D_i^{(k)}, \qquad \mathrm{PD}_k^{\mathrm{PiT}} = \mathbb{P}\left(D_{i,t}^{(k)} = 1 \,\middle|\, \boldsymbol{X}_i, \boldsymbol{Z}_u\right),$$

so section 2.4's flag is the $t = 0$ case and the worst-ever aggregation carries over untouched. The PiT PD conditions on the macroeconomic state $\boldsymbol{Z}_u$ in the calendar month the window opens, $u = g_i + t$, rather than on u itself, since a future month carries no observations of its own while the macro variables are forecastable. A TTC PD is indeed an average of default rates, and the series being averaged is the portfolio one, which Botha and Verster (2025) put on this lecture's axes define as

$$\mathrm{DR}_u^{(k)} = \frac{1}{n_u} \sum_{i \in \mathcal{P}_u} D_{i,\,u-g_i}^{(k)}, \qquad \mathcal{P}_u = \{i : \text{on book and performing at } u\}, \qquad n_u = \#\mathcal{P}_u.$$

The paper writes D_t, $\mathcal{S}_P(t)$ and n_t' with t as calendar time, so anyone going to the source should carry that translation; here the calendar axis has to be u, since t is months on book everywhere else. Regressing $\mathrm{DR}_u^{(k)}$ on $\boldsymbol{Z}_u$ builds the PiT PD and averaging it over u gives the regulatory TTC estimator.

The twelve-month cross-section modelled in this lecture cannot yield a PiT PD, because conditional on t the vintage g_i and the calendar month u are collinear. That constraint belongs to the table rather than to the book: expanding the same loans to one row per month at risk gives Estonia 150 distinct calendar months and identifies a macro coefficient. Credit lecture R1 does that, and it also carries the two conditioning axes hiding inside the word "conditional", the Jensen trap that makes averaging a one-way operation, and the eleven competing estimators.

WHAT A HYBRID PD LOOKS LIKE IN FORMULAS

Two representations, of which the second is the one production systems actually parameterise.

The blunt one interpolates on the logit scale between the two extremes,

$$\operatorname{logit} \mathrm{PD}_k^{\mathrm{hyb}} = (1-\lambda) \operatorname{logit} \mathrm{PD}_k^{\mathrm{TTC}}\left(\boldsymbol{X}_i\right) + \lambda \operatorname{logit} \mathrm{PD}_k^{\mathrm{PiT}}\left(\boldsymbol{X}_i, \boldsymbol{Z}_u\right), \qquad \lambda \in [0,1],$$

with $\lambda = 0$ fully through the cycle and $\lambda = 1$ fully point in time. It leaves λ with no meaning beyond itself, which is why a validator will ask how it was set.

The one-factor version gives the dial a meaning, writing $Z_u \sim \mathcal{N}(0,1)$ for a systematic factor in calendar month u, high values benign, and $\rho \in [0,1)$ for the loan's loading on it:

$$\mathrm{PD}_k^{\mathrm{hyb}}\left(\boldsymbol{X}_i, Z_u\right) = \Phi\left(\frac{\Phi^{-1}\left(\mathrm{PD}_k^{\mathrm{TTC}}\left(\boldsymbol{X}_i\right)\right) - \sqrt{\rho}\, Z_u}{\sqrt{1-\rho}}\right).$$

This is Vasicek's conditional default probability, and ρ has replaced λ as the dial, measuring how far the same input swings with the cycle.

Both of its useful properties belong to lecture R2. Averaging the output over Z returns the TTC PD **exactly**, and stressing the factor instead, at $Z_u = -\Phi^{-1}(0.999)$, turns the same expression into the IRB capital formula with ρ prescribed by exposure class rather than fitted. R2 takes that substitution through the whole production sequence, from this scorecard to a risk weight.

2.7 Regression modelling

In general, the true data generating mechanism is unknown. Therefore, one cannot compute the regression function $\boldsymbol{X} \mapsto \mu_k(\boldsymbol{X})$, and one needs to estimate it from past data

$$\mathcal{L} = \left(D_i^{(k)}, \boldsymbol{X}_i \right)_{i \in \mathcal{W}_k}.$$

$\mathcal{L}$ is called the **learning sample**, and it is indexed by the retained set $\mathcal{W}_k$ of the previous section. The two letters had to be separated: the original lecture uses $\mathcal{L}$ for the learning sample throughout, so the retention set cannot share it. Before solving this estimation problem, one should always empirically study the available data. This empirical study is crucial to fully understand the problem and to select a suitable class of candidate regression models $\mathcal{M} = \{\mu\}$.

3 Empirical data analysis

We start by classifying the loans w.r.t. their default outcome.

```
dat.group_by("default_12m").agg(pl.len().alias("loans")).sort("default_12m")
```

shape: (2, 2)

default_12m	loans
i8	u32
0	105676
1	43057

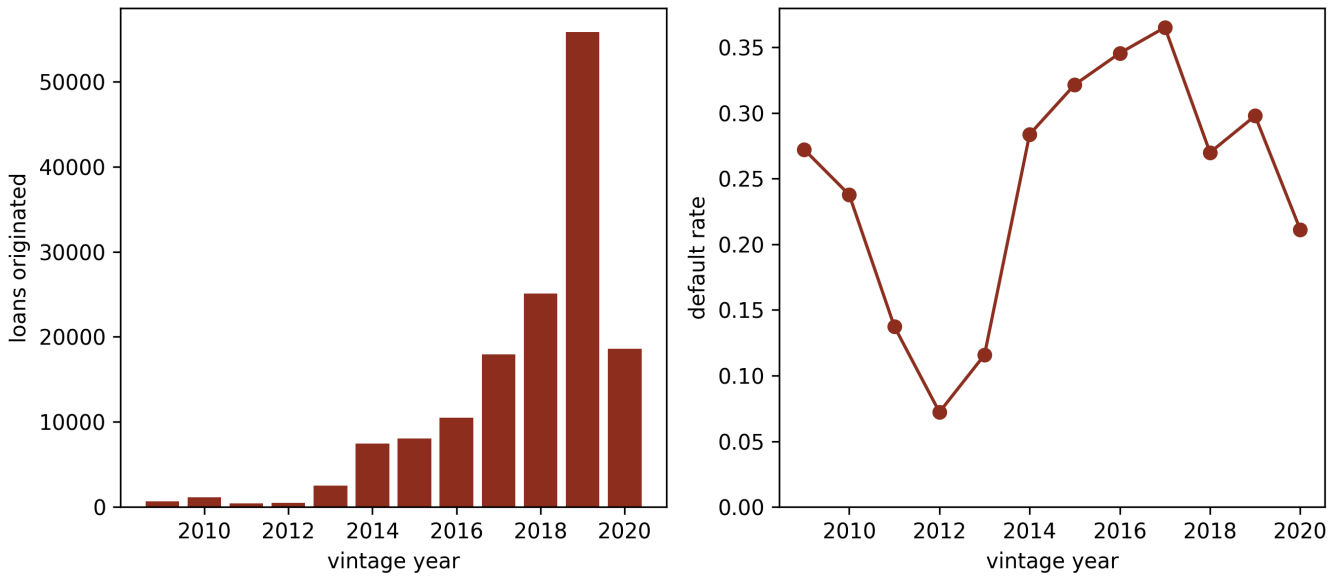
```
print(f"empirical default rate: {dat['default_12m'].mean():.2%}")
```

```
empirical default rate: 28.95%
```

The empirical twelve-month default rate is 28.95 per cent. Here the credit example diverges from the insurance one on purpose. The French MTPL portfolio has an empirical claims frequency of 7.36 per cent, and the original lecture frames this as a **class imbalance problem**, i.e., insurance claims are rare events. Bondora is a subprime peer-to-peer book, and at this default rate the event is anything but rare. Rarity is a property of the portfolio rather than of credit risk as such. A prime credit card book behaves like the insurance case; the companion dataset `credit_card_dev.parquet` in this repository, an anonymised bank-grade card portfolio, has a bad rate of 1.42 per cent. Class imbalance will therefore return with force when that dataset takes the stage in a later lecture.

A second structural feature deserves a look before any covariate does. The book grew by two orders of magnitude over its life, and the default rate moved with it.

```
vint = (
    dat.with_columns(vintage=pl.col("LoanDate").dt.year())
    .group_by("vintage")
    .agg(pl.len().alias("loans"), pl.col("default_12m").mean().alias("rate"))
    .sort("vintage")
)
fig, axs = plt.subplots(1, 2)
axs[0].bar(vint["vintage"], vint["loans"], color="#8C2F1E")
axs[0].set_xlabel("vintage year"); axs[0].set_ylabel("loans originated")
axs[1].plot(vint["vintage"], vint["rate"], "o-", color="#8C2F1E")
axs[1].set_xlabel("vintage year"); axs[1].set_ylabel("default rate")
axs[1].set_ylim(0, None)
plt.tight_layout(); plt.show()
```



(lhs): Loans originated per vintage year. (rhs): Empirical twelve-month default rate per vintage year. The portfolio is strongly non-stationary in both volume and risk.

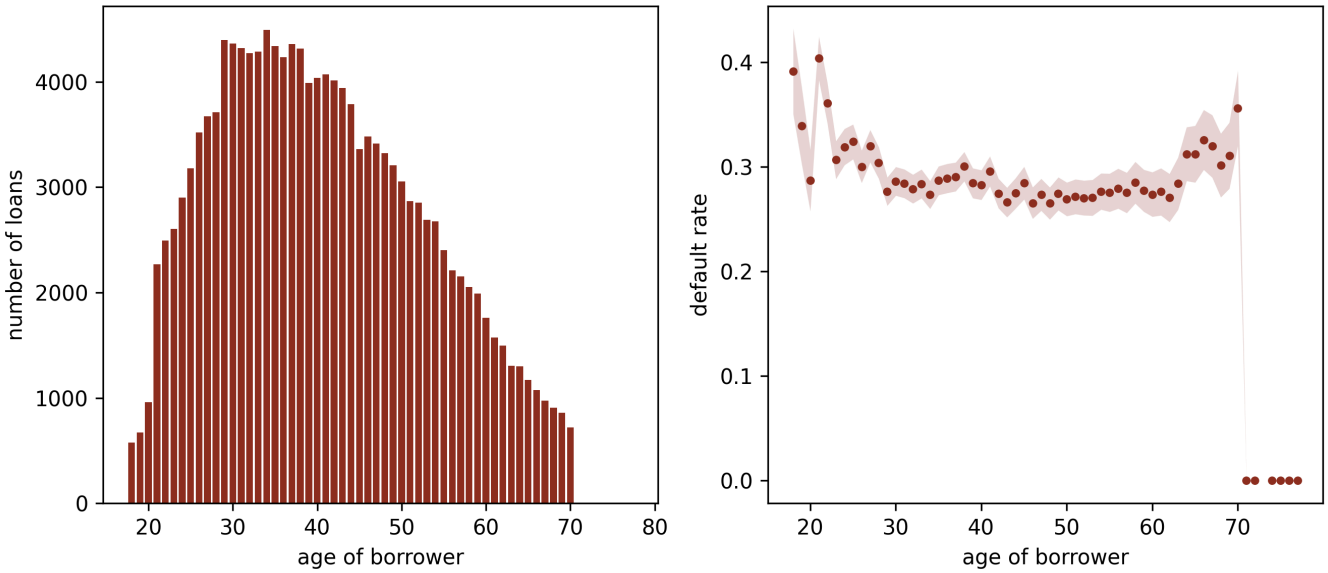
The default rate runs from 7 per cent in the 2012 vintage to 37 per cent in 2017. Insurance portfolios drift too, but rarely this violently; the drift here mixes platform growth, underwriting changes and country expansion. We ignore the time dimension for the rest of this lecture and model the pooled book, as the original lecture pools its single accident year. The vintage axis returns when we need an honest out-of-time validation split.

For the covariate analysis we consider two covariates, `Age` and log-`IncomeTotal`, playing the roles that `DrivAge` and log-`Density` play in the insurance lecture. Later, all covariates are considered. We remove the 53 loans with a cleaned age of null and the 63 loans declaring zero income, which leaves the modelling sample used from here on.

```
model_dat = (
    dat.filter(pl.col("Age").is_not_null() & (pl.col("IncomeTotal") > 0))
    .with_columns(logIncome=pl.col("IncomeTotal").log())
)
print(f"modelling sample: n = {model_dat.height:,}")
```

```
modelling sample: n = 148,670
```

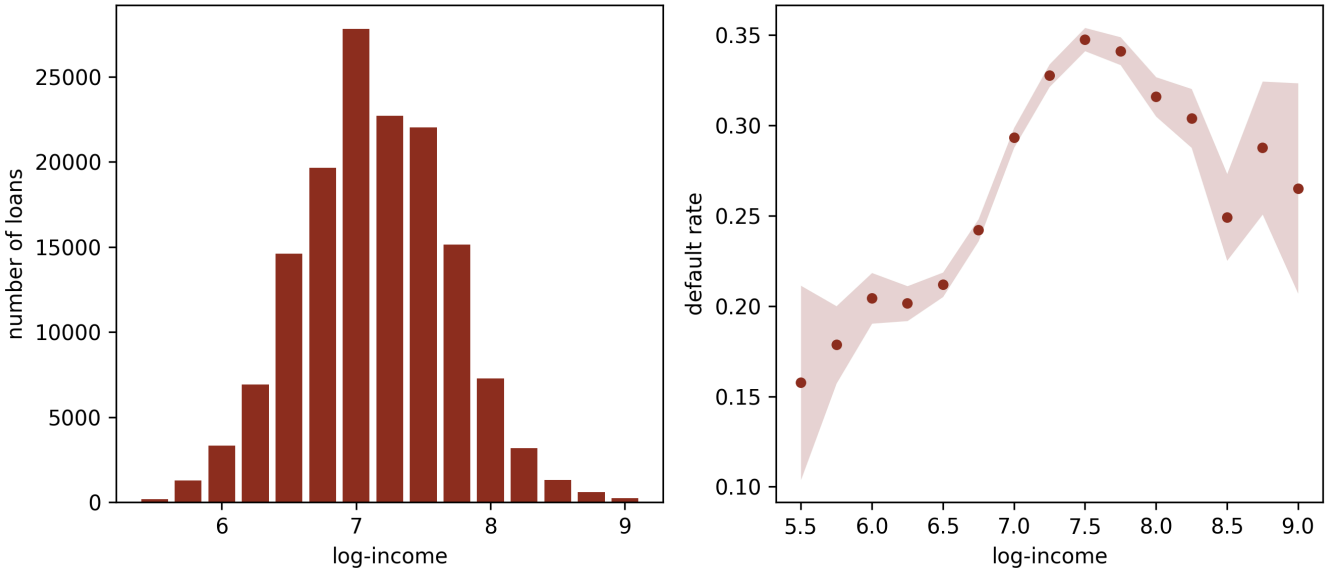
```
def rate_by(df, col):
    g = (
        df.group_by(col)
        .agg(pl.len().alias("n"), pl.col("default_12m").mean().alias("rate"))
        .sort(col)
    )
    se = (g["rate"] * (1 - g["rate"]) / g["n"]).sqrt()
    return g["n"], g[col], g["rate"], se

n_a, age, r_a, se_a = rate_by(model_dat, "Age")
fig, axs = plt.subplots(1, 2)
axs[0].bar(age, n_a, color="#8C2F1E")
axs[0].set_xlabel("age of borrower"); axs[0].set_ylabel("number of loans")
axs[1].plot(age, r_a, "o", ms=3, color="#8C2F1E")
axs[1].fill_between(age, r_a - 2 * se_a, r_a + 2 * se_a,
                    color="#8C2F1E", alpha=0.2, linewidth=0)
axs[1].set_xlabel("age of borrower"); axs[1].set_ylabel("default rate")
plt.tight_layout(); plt.show()
```



(lhs): Number of loans per borrower age. (rhs): Empirical default rate per borrower age; the bounds correspond to 2 standard deviations in a Bernoulli model. Ages with fewer than 200 loans are pooled into their neighbours by the eye; the bounds widen accordingly.

The age profile is **non-monotone**. Default risk peaks around the mid-twenties at roughly 32 per cent, eases to about 27 per cent through middle age, and climbs again beyond 65, where declared pension income replaces salary. The same shape, a hump for the young and a deterioration at the old end, drove the insurance lecture's discussion of `DrivAge`, and it will defeat the same naive model below.

```
model_dat = model_dat.with_columns(
    incBand=(pl.col("logIncome") * 4).round(0) / 4
)
n_b, band, r_b, se_b = rate_by(model_dat.filter(
    pl.col("logIncome").is_between(5.5, 9.0)), "incBand")
fig, axs = plt.subplots(1, 2)
axs[0].bar(band, n_b, width=0.2, color="#8C2F1E")
axs[0].set_xlabel("log-income"); axs[0].set_ylabel("number of loans")
axs[1].plot(band, r_b, "o", ms=4, color="#8C2F1E")
axs[1].fill_between(band, r_b - 2 * se_b, r_b + 2 * se_b,
                    color="#8C2F1E", alpha=0.2, linewidth=0)
axs[1].set_xlabel("log-income"); axs[1].set_ylabel("default rate")
plt.tight_layout(); plt.show()
```

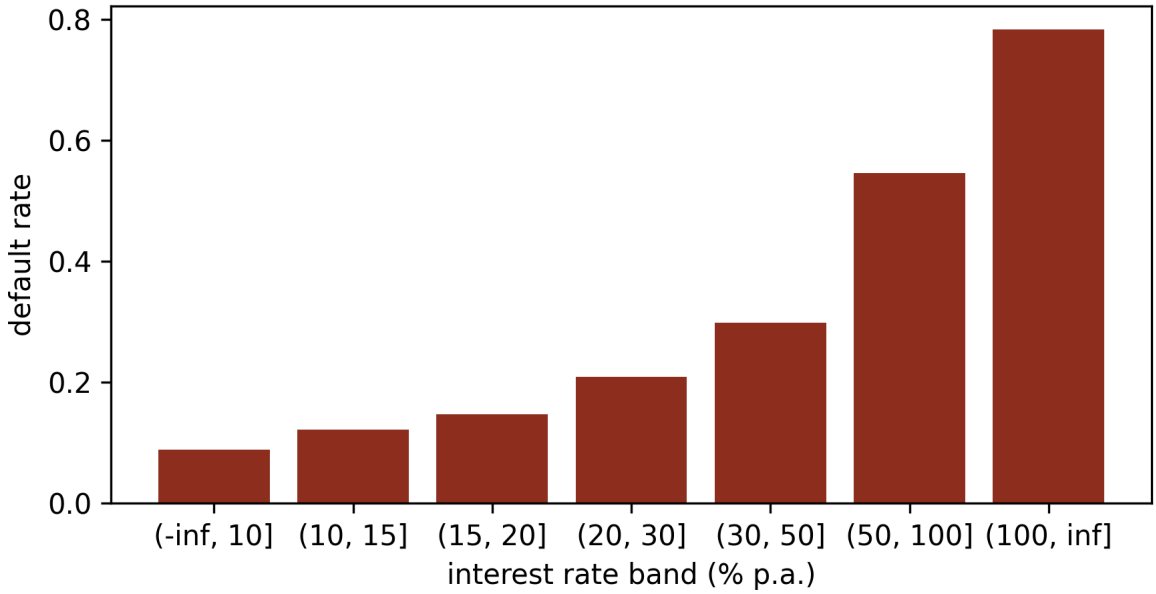


(lhs): Number of loans per log-income band. (rhs): Empirical default rate per log-income band; the bounds correspond to 2 standard deviations in a Bernoulli model.

The income profile should worry any credit risk modeller on sight. Default risk **rises** with declared income across most of the range, from below 20 per cent at the bottom to peaks above 34 per cent, before easing at the very top. No underwriter believes that richer borrowers are intrinsically worse risks, so something else must be driving the marginal curve. We hold that thought for a moment and look at one more covariate first.

An aside on the price of risk. The contractual interest rate is the platform's own answer to this prediction problem, set at listing from its rating model. Its marginal profile is brutally clean.

```
ib = (
    model_dat.with_columns(band=pl.col("Interest").cut([10, 15, 20, 30, 50, 100]))
    .group_by("band").agg(pl.len().alias("n"), pl.col("default_12m").mean().alias("rate"))
    .sort("band")
)
fig, ax = plt.subplots(figsize=(6, 3.2))
ax.bar([str(b) for b in ib["band"]], ib["rate"], color="#8C2F1E")
ax.set_xlabel("interest rate band (% p.a.)"); ax.set_ylabel("default rate")
plt.tight_layout(); plt.show()
```



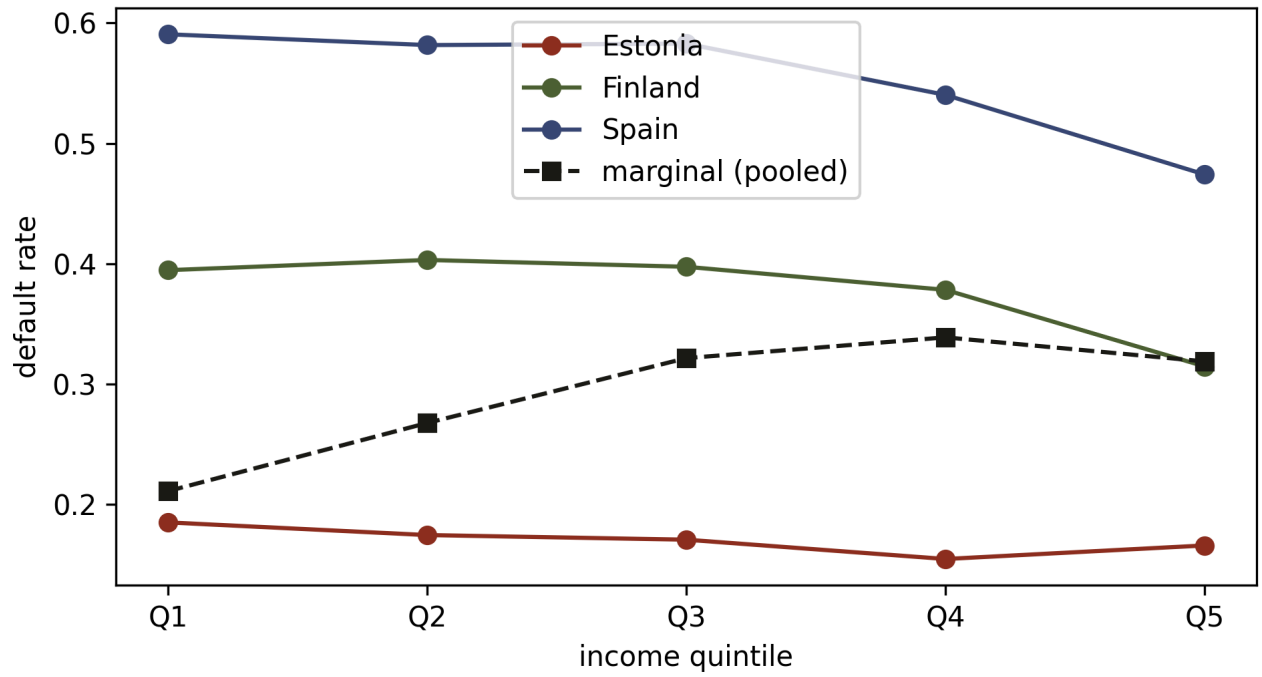
Empirical default rate per contractual interest band. Interest is the platform's price of risk, so its gradient is the platform's own PD model echoed back.

The gradient runs from 9 per cent below a 10 per cent coupon to 78 per cent above a 100 per cent coupon. As a covariate in a PD model, however, `Interest` is **endogenous**. It is an output of the lender's previous risk assessment rather than a property of the borrower, and a model built on it mostly recovers the incumbent model. We therefore leave it out of the regressions below and keep it as a benchmark of how much structure a good model should find. In causal language the rate is a **mediator**, sitting on the path from the borrower's characteristics to default, and it is simultaneously a **collider** on a second path, since the platform's unobserved view of the borrower points into both the rate and the outcome. Credit lecture C1 draws both paths and measures what conditioning on the rate costs: the income coefficient barely moves while a third of the estimated Spanish country effect disappears into the coupon.

3.1 Covariates interact

Generally, covariates **interact**, and one cannot simply combine marginal default rates to obtain a correct PD. The insurance lecture argues this with a hypothetical, i.e., young drivers may cluster in dense cities, so the two marginal penalties may double-count one risk factor. The Bondora book turns the hypothetical into a demonstration. The three main countries have very different income levels and very different risk levels, and the composition generates the rising marginal income curve above.

```
countries = {"EE": "Estonia", "FI": "Finland", "ES": "Spain"}
fig, ax = plt.subplots(figsize=(6.5, 3.6))
for code, colour in zip(countries, ["#8C2F1E", "#4F6333", "#3B4A75"]):
    sub = model_dat.filter(pl.col("Country") == code)
    qs = np.quantile(sub["logIncome"], [0.2, 0.4, 0.6, 0.8])
    banded = sub.with_columns(
        q=pl.col("logIncome").cut(list(qs), labels=["Q1", "Q2", "Q3", "Q4", "Q5"])
    )
    curve = banded.group_by("q").agg(pl.col("default_12m").mean()).sort("q")
    ax.plot(curve["q"], curve["default_12m"], "o-", label=countries[code], color=colour)
qs = np.quantile(model_dat["logIncome"], [0.2, 0.4, 0.6, 0.8])
marg = (
    model_dat.with_columns(
        q=pl.col("logIncome").cut(list(qs), labels=["Q1", "Q2", "Q3", "Q4", "Q5"]))
    .group_by("q").agg(pl.col("default_12m").mean()).sort("q")
)
ax.plot(marg["q"], marg["default_12m"], "s--", color="#1B1A17", label="marginal (pooled)")
ax.set_xlabel("income quintile"); ax.set_ylabel("default rate"); ax.legend()
plt.tight_layout(); plt.show()
```

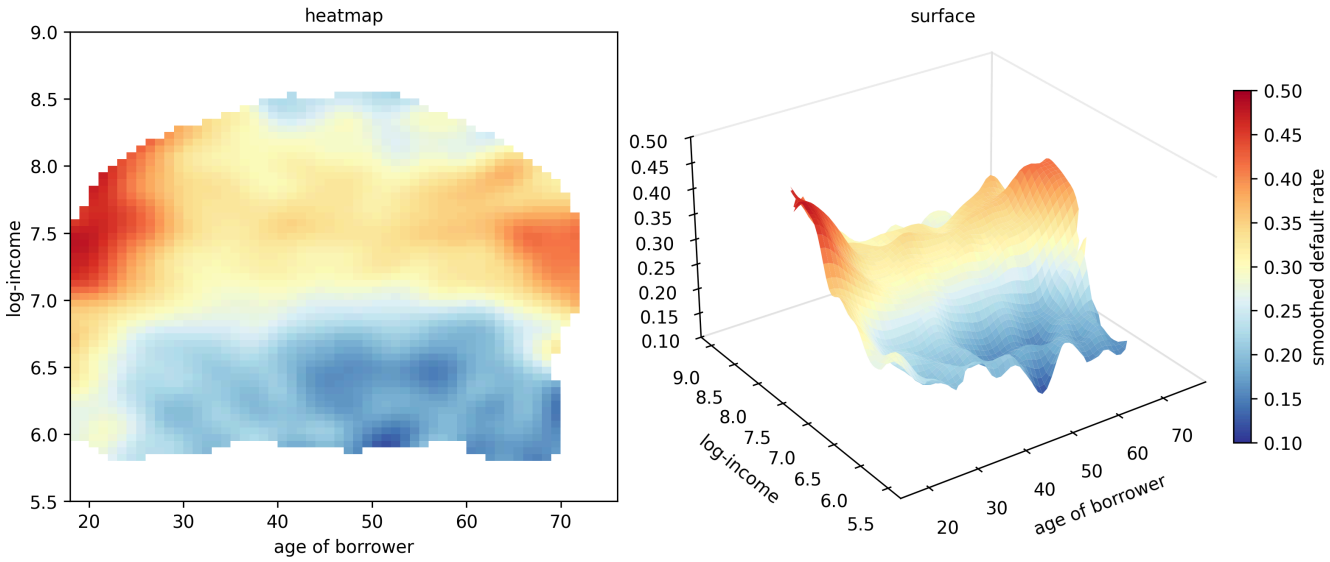


Empirical default rate per within-country income quintile (Q1 low to Q5 high), by country and marginally. Within every country the rate falls with income; marginally it rises. The marginal curve is composition, i.e., Simpson's paradox in the raw data.

Within Estonia, Finland and Spain separately, default risk **falls** with income, exactly as underwriting intuition demands. Pooled, it rises, because Finland and Spain combine higher nominal incomes with default rates of 38 and 55 per cent against Estonia's 17. The marginal income effect is a country effect wearing a disguise. Multiplying marginal profiles would price this book absurdly, and the same mechanism, in gentler form, is why the insurance lecture insists on estimating $\mu(\boldsymbol{X})$ **jointly** in the components of $\boldsymbol{X} = (X_1, \dots, X_q)^\top$ rather than marginally.

The mechanism has a name. `Country` is a **confounder** of income and default, being a common cause of both, so the pooled gradient carries an open path that runs from income back through country and on to default. Conditioning on country blocks it, which is exactly what GLM3 does below. Credit lecture C1 takes that structure apart: it separates interaction from effect modification, derives an adjustment set from the graph rather than from predictive lift, and recovers the marginal income curve this figure cannot supply by averaging the country-conditional model over the observed country distribution. Standardised that way the curve **falls**, from 29.6 to 27.7 per cent across a fourfold income range, where the raw pooled curve above rises.

```
from scipy.ndimage import gaussian_filter
from matplotlib.colors import Normalize

x = model_dat["Age"].to_numpy().astype(float)
y = model_dat["logIncome"].to_numpy()
d = model_dat["default_12m"].to_numpy().astype(float)

x_edges = np.linspace(18, 76, 59)
y_edges = np.linspace(5.5, 9.0, 71)
cnt, _, _ = np.histogram2d(x, y, bins=[x_edges, y_edges])
dft, _, _ = np.histogram2d(x, y, bins=[x_edges, y_edges], weights=d)
cnt_s = gaussian_filter(cnt, sigma=2.0)
dft_s = gaussian_filter(dft, sigma=2.0)
rate = np.where(cnt_s > 8, dft_s / np.maximum(cnt_s, 1e-9), np.nan)

# Both panels are drawn from this one array on this one scale, so the surface
# cannot disagree with the heatmap. Bin centres, not edges, carry the surface.
x_c = 0.5 * (x_edges[:-1] + x_edges[1:])
y_c = 0.5 * (y_edges[:-1] + y_edges[1:])
X_g, Y_g = np.meshgrid(x_c, y_c, indexing="ij")   # both (58, 70), as rate is
norm = Normalize(vmin=0.1, vmax=0.5)

fig = plt.figure(figsize=(10.5, 4.4), layout="constrained")
ax = fig.add_subplot(1, 2, 1)
im = ax.imshow(
    rate.T, origin="lower", aspect="auto",
    extent=(18, 76, 5.5, 9.0), cmap="RdYlBu_r", norm=norm)
ax.set_xlabel("age of borrower"); ax.set_ylabel("log-income")
ax.set_title("heatmap", fontsize=10)

ax3 = fig.add_subplot(1, 2, 2, projection="3d")
ax3.plot_surface(
    X_g, Y_g, rate, cmap="RdYlBu_r", norm=norm,
    rcount=58, ccount=70, linewidth=0, edgecolor="none", antialiased=True)
# The z axis carries no label, since the colourbar already names the quantity and
# a zlabel here collides with the heatmap panel's right edge.
ax3.set_xlabel("age of borrower"); ax3.set_ylabel("log-income")
ax3.set_zlim(0.1, 0.5)
ax3.view_init(elev=28, azim=-125)
ax3.set_box_aspect((1, 1, 0.62), zoom=1.18)
ax3.set_title("surface", fontsize=10)
for pane in (ax3.xaxis.pane, ax3.yaxis.pane, ax3.zaxis.pane):
    pane.set_facecolor("white"); pane.set_edgecolor("0.85")
ax3.grid(False)

fig.colorbar(im, ax=(ax, ax3), shrink=0.75, label="smoothed default rate")
plt.show()
```



Kernel-smoothed empirical default rate as a function of borrower age and log-income, shown twice off one smoothed array. Both panels share the colour scale 0.10 to 0.50 and the single colourbar, so a colour means the same default rate in each, and on the surface the height encodes that same rate as well as the colour. White space in the heatmap, and the gaps in the surface, mark covariate combinations too thin in the portfolio to estimate (fewer than eight effective loans after smoothing). The heatmap is the better panel for reading a value off a pair of covariates; the surface is the better panel for seeing that the joint shape is not additive, since the age profile changes its form as log-income moves rather than merely shifting up or down. Read the surface with age increasing to the right along the front edge and log-income increasing away from the viewer to the left, so its income axis runs in the opposite visual sense to the heatmap's vertical one.

Both panels show the empirical default rate as a function of the two covariates jointly, smoothed with a Gaussian kernel; the original lecture uses loess smoothing for the same purpose. They are two views of one array on one colour scale, so they carry the same numbers and differ only in what they make easy to see. The heatmap is the panel to read a rate off a pair of covariates, since position and colour are both undistorted. The surface adds the one thing a flat map hides, namely the height, and with it a visual test of additivity. Were the joint effect additive, every slice along age would be the same curve, raised or lowered by the income level alone. Instead the age profile is pronounced at high incomes, where the rate runs at its highest below age 25, falls away through the middle ages and lifts again past 65, whereas at low incomes it is close to flat. Age therefore matters far more to a high-income borrower in this book than to a low-income one, and no amount of tilting a plane will reproduce that. Either way the colouring shows quite some non-linear structure, and it does not support any simple combination of the two marginal profiles. This requires regression modelling or machine learning solutions, e.g.,

- generalised linear models (GLMs)
- neural networks and deep learning
- regression trees
- gradient boosting machines (GBMs)

4 A generalised linear model example

We start with generalised linear models (GLMs) for the class $\mathcal{M}$ of considered regression functionals $\boldsymbol{X} \mapsto \mu(\boldsymbol{X})$. The underlying mathematical theory is presented in careful detail in the next lecture. For a binary response, the canonical choice is the **Bernoulli GLM with logit link**, i.e., logistic regression, the workhorse of credit scoring. The simplest setting considers

$$\boldsymbol{X} \mapsto \mu(\boldsymbol{X}) = \frac{1}{1 + \exp\left(-\beta_0 - \beta_1 X_1 - \beta_2 X_2\right)},$$

for $\boldsymbol{X} = (X_1, X_2) = (\text{Age}, \text{log-Income})$ and parameter $(\beta_0, \beta_1, \beta_2)$. Where the insurance lecture's log-link produced a model multiplicative in the **frequency**, the logit link produces a model multiplicative in the **odds** $\mu/(1-\mu)$; each covariate scales the odds

of default by a constant factor per unit.

4.0.1 Model GLM1 (Bernoulli with canonical logit link)

```
import pandas as pd
import statsmodels.formula.api as smf
import statsmodels.api as sm

glm_dat = model_dat.select(
    ["default_12m", "Age", "logIncome", "Country"]).to_pandas()

glm1 = smf.glm("default_12m ~ Age + logIncome",
               data=glm_dat, family=sm.families.Binomial()).fit()
print(glm1.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:            default_12m   No. Observations:               148670
Model:                            GLM   Df Residuals:                   148667
Model Family:                Binomial   Df Model:                            2
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -89001.
Date:                Thu, 03 Sep 2026   Deviance:                   1.7800e+05
Time:                        10:21:52   Pearson chi2:                 1.49e+05
No. Iterations:                     5   Pseudo R-squ. (CS):           0.006011
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -2.5133      0.066    -38.345      0.000      -2.642      -2.385
Age           -0.0053      0.000    -11.311      0.000      -0.006      -0.004
logIncome      0.2550      0.009     28.520      0.000       0.238       0.273
==============================================================================
```

Both coefficients are highly significant, and both should be distrusted. The income coefficient is **positive**, faithfully reproducing the confounded marginal curve of the previous section, and the single linear age term cannot represent a hump at 25 together with a deterioration after 65. In case of non-monotone behaviour, one can build categorical classes. We build the age classes as follows (implemented by dummy coding, explained in the Wednesday lecture); relative to the insurance lecture's cuts we split the 51 to 70 band, because our non-monotonicity also sits at the old end.

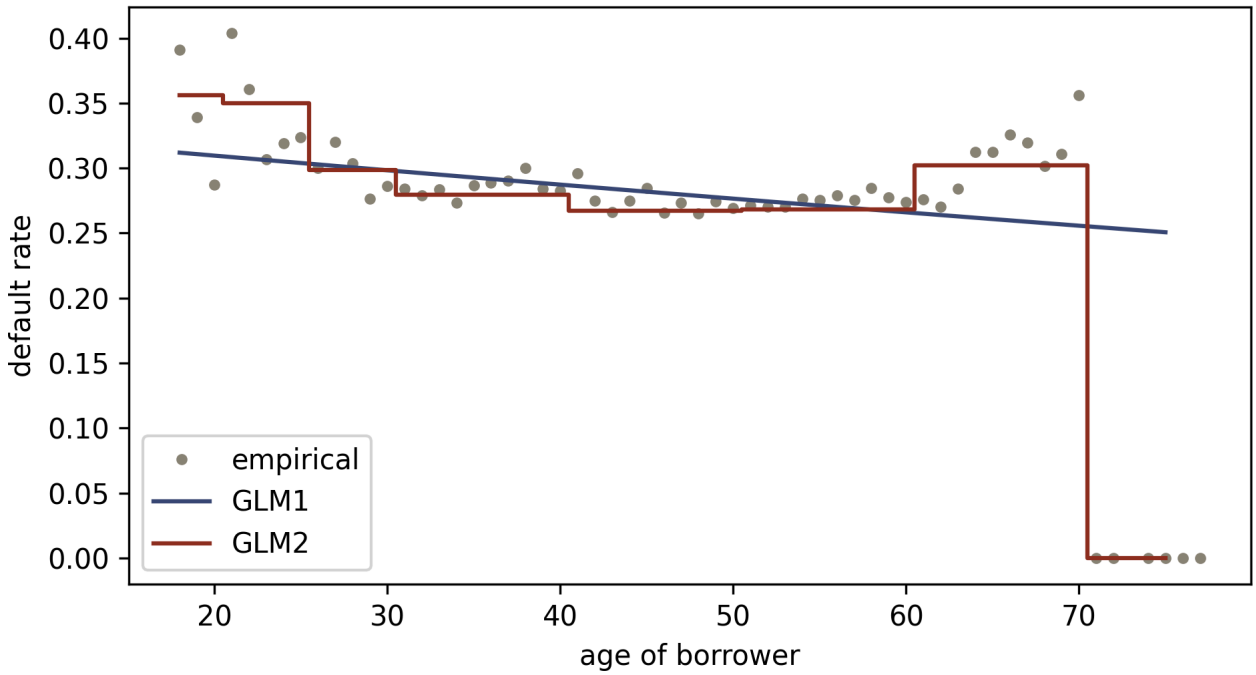
4.0.2 Model GLM2 (categorical age classes)

```
glm_dat["AgeClass"] = pd.cut(
    glm_dat["Age"], [18, 20, 25, 30, 40, 50, 60, 70, 101],
    labels=["18-20", "21-25", "26-30", "31-40", "41-50",
            "51-60", "61-70", "71+"],
    include_lowest=True)

glm2 = smf.glm("default_12m ~ AgeClass + logIncome",
               data=glm_dat, family=sm.families.Binomial()).fit()
print(glm2.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:            default_12m   No. Observations:               148670
Model:                            GLM   Df Residuals:                   148661
Model Family:                Binomial   Df Model:                            8
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -88846.
Date:                Thu, 03 Sep 2026   Deviance:                   1.7769e+05
Time:                        10:21:52   Pearson chi2:                 1.49e+05
No. Iterations:                    21   Pseudo R-squ. (CS):           0.008074
Covariance Type:            nonrobust
================================================================================
                       coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------
Intercept           -2.5564      0.076    -33.655      0.000      -2.705      -2.408
AgeClass[T.21-25]   -0.0258      0.049     -0.526      0.599      -0.122       0.070
AgeClass[T.26-30]   -0.2622      0.048     -5.440      0.000      -0.357      -0.168
AgeClass[T.31-40]   -0.3535      0.047     -7.527      0.000      -0.445      -0.261
AgeClass[T.41-50]   -0.4157      0.047     -8.795      0.000      -0.508      -0.323
AgeClass[T.51-60]   -0.4114      0.048     -8.567      0.000      -0.505      -0.317
AgeClass[T.61-70]   -0.2444      0.050     -4.886      0.000      -0.342      -0.146
AgeClass[T.71+]    -21.9977   1.32e+04     -0.002      0.999   -2.58e+04    2.58e+04
logIncome            0.2754      0.009     30.461      0.000       0.258       0.293
================================================================================
```

This estimates an individual regression parameter β_j for each age class, and the fit improves accordingly. Because of the logit link, the model has a multiplicative structure in the odds; e.g., a borrower in the `71+` class carries the same income effect as every other borrower, scaled by their own age-class odds factor.

```
li_med = float(np.median(glm_dat["logIncome"]))
grid = pd.DataFrame({"Age": np.arange(18, 76), "logIncome": li_med})
grid["AgeClass"] = pd.cut(
    grid["Age"], [18, 20, 25, 30, 40, 50, 60, 70, 101],
    labels=["18-20", "21-25", "26-30", "31-40", "41-50",
            "51-60", "61-70", "71+"],
    include_lowest=True)

fig, ax = plt.subplots(figsize=(6.5, 3.6))
ax.plot(age, r_a, "o", ms=3, color="#8A8377", label="empirical")
ax.plot(grid["Age"], glm1.predict(grid), color="#3B4A75", label="GLM1")
ax.step(grid["Age"], glm2.predict(grid), color="#8C2F1E", label="GLM2", where="mid")
ax.set_xlabel("age of borrower"); ax.set_ylabel("default rate"); ax.legend()
plt.tight_layout(); plt.show()
```



Empirical default rate per age (dots) against the GLM1 and GLM2 age profiles, both evaluated at the median log-income. GLM1 forces monotonicity; GLM2 recovers the hump at young ages and the rise after 65.

GLM2 reflects the non-monotonicity in age. However, the income coefficient still carries the wrong sign, because neither model has been told about the country composition. The insurance lecture stops at two covariates and lets the remaining structure motivate richer models; we allow ourselves one more step, because it lands the interaction argument of the previous section.

4.0.3 Model GLM3 (adding the confounder)

```
glm3 = smf.glm("default_12m ~ AgeClass + logIncome + Country",
               data=glm_dat, family=sm.families.Binomial()).fit()
print(glm3.summary().tables[1])
```

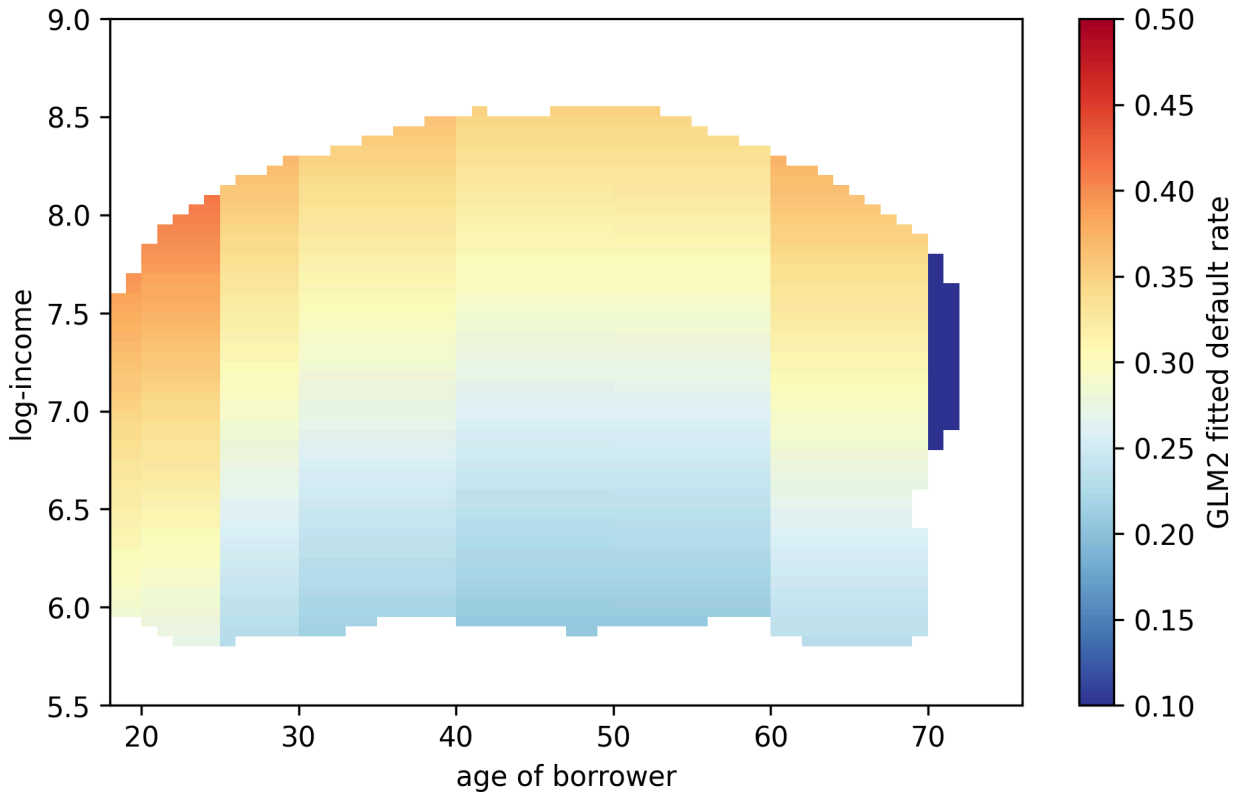
```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -0.3755      0.087     -4.335      0.000      -0.545      -0.206
AgeClass[T.21-25]    -0.1984      0.051     -3.914      0.000      -0.298      -0.099
AgeClass[T.26-30]    -0.5417      0.050    -10.856      0.000      -0.640      -0.444
AgeClass[T.31-40]    -0.7513      0.049    -15.450      0.000      -0.847      -0.656
AgeClass[T.41-50]    -0.8636      0.049    -17.619      0.000      -0.960      -0.767
AgeClass[T.51-60]    -0.9240      0.050    -18.530      0.000      -1.022      -0.826
AgeClass[T.61-70]    -0.8126      0.052    -15.610      0.000      -0.915      -0.711
AgeClass[T.71+]     -21.9170   1.28e+04     -0.002      0.999   -2.52e+04    2.52e+04
Country[T.ES]         1.9054      0.016    118.879      0.000       1.874       1.937
Country[T.FI]         1.2538      0.017     74.856      0.000       1.221       1.287
Country[T.SK]         2.4625      0.129     19.148      0.000       2.210       2.715
logIncome     -0.0778      0.011     -7.064      0.000      -0.099      -0.056
==============================================================================
```

```
comparison = pd.DataFrame({
    "deviance": [glm1.deviance, glm2.deviance, glm3.deviance],
    "AIC": [glm1.aic, glm2.aic, glm3.aic],
    "income coefficient": [
        glm1.params["logIncome"], glm2.params["logIncome"],
        glm3.params["logIncome"]],
}, index=["GLM1", "GLM2", "GLM3"])
comparison.round(3)
```

	deviance	AIC	income coefficient
GLM1	178001.582	178007.582	0.255
GLM2	177692.707	177710.707	0.275
GLM3	161372.056	161396.056	-0.078

Conditioning on country flips the income coefficient negative, i.e., within a country, higher income now reduces the odds of default, exactly as the quintile curves showed and as underwriting intuition demands. The wrongly signed coefficient was evidence about the portfolio's composition all along, and it had to be explained before the model could be believed. This is worth carrying through the whole lecture series, because deep learning models absorb such composition effects silently, and in credit they must be found and explained before a model is used.

```
gx = (x_edges[:-1] + x_edges[1:]) / 2
gy = (y_edges[:-1] + y_edges[1:]) / 2
gxx, gyy = np.meshgrid(gx, gy, indexing="ij")
gframe = pd.DataFrame({"Age": gxx.ravel(), "logIncome": gyy.ravel()})
gframe["AgeClass"] = pd.cut(
    gframe["Age"], [18, 20, 25, 30, 40, 50, 60, 70, 101],
    labels=["18-20", "21-25", "26-30", "31-40", "41-50",
            "51-60", "61-70", "71+"],
    include_lowest=True)
glm2_surface = np.where(
    np.isnan(rate), np.nan,
    glm2.predict(gframe).to_numpy().reshape(gxx.shape))

fig, ax = plt.subplots(figsize=(6.5, 4.2))
im = ax.imshow(glm2_surface.T, origin="lower", aspect="auto",
               extent=(18, 76, 5.5, 9.0), cmap="RdYlBu_r", vmin=0.1, vmax=0.5)
ax.set_xlabel("age of borrower"); ax.set_ylabel("log-income")
fig.colorbar(im, label="GLM2 fitted default rate")
plt.tight_layout(); plt.show()
```



Fitted default rate of GLM2 over the (age, log-income) grid, masked to the covariate combinations present in the portfolio. The multiplicative odds structure produces smooth vertical and horizontal banding and, by construction, no interaction between the two axes.

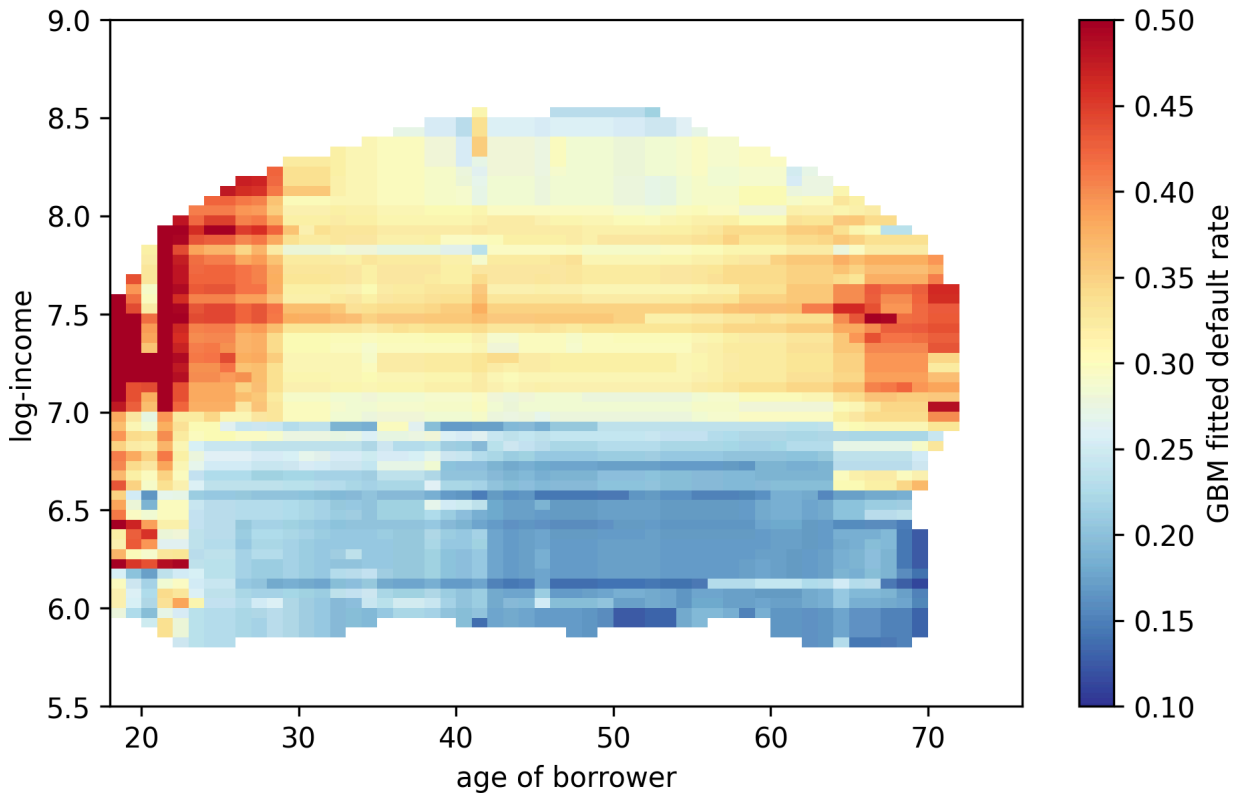
Comparing this surface with the empirical heatmap, the GLM reproduces the main gradients but none of the diagonal structure, which is a motivation to consider more complex regression models.

5 Gradient boosting machine

As a first machine learning regression model, we present a gradient boosting machine (GBM). GBMs belong to the most powerful techniques on tabular data, in particular the powerful versions like XGBoost, LightGBM or CatBoost; on credit bureau data they

remain the benchmark to beat in most published comparisons. A GBM partitions the covariate space into different (homogeneous) subsets and estimates the default rate on each subset. We fit scikit-learn's histogram-based implementation on the two selected covariates and only present the resulting heatmap; GBMs are not further studied in this series.

```
from sklearn.ensemble import HistGradientBoostingClassifier

gbm = HistGradientBoostingClassifier(
    max_iter=300, learning_rate=0.05, max_leaf_nodes=15, random_state=1)
gbm.fit(np.column_stack([x, y]), d)

gbm_surface = np.where(
    np.isnan(rate), np.nan,
    gbm.predict_proba(np.column_stack([gxx.ravel(), gyy.ravel()]))[:, 1]
    .reshape(gxx.shape))

fig, ax = plt.subplots(figsize=(6.5, 4.2))
im = ax.imshow(gbm_surface.T, origin="lower", aspect="auto",
               extent=(18, 76, 5.5, 9.0), cmap="RdYlBu_r", vmin=0.1, vmax=0.5)
ax.set_xlabel("age of borrower"); ax.set_ylabel("log-income")
fig.colorbar(im, label="GBM fitted default rate")
plt.tight_layout(); plt.show()
```



Fitted default rate of the GBM over the (age, log-income) grid, same masking and colour scale as before. The GBM recovers the diagonal structure of the empirical heatmap, at the price of a piecewise-constant, non-smooth surface.

The GBM seems to reflect the observed (smoothed) data very well. The criticisms one may raise are that the resulting regression function is not smooth and that it cannot easily be extrapolated, e.g., to incomes beyond the observed range. A crucial question is whether this GBM only extracts the structural (systematic) part or whether it also reflects some of the noise in the observed data. The latter would be harmful, because the noisy part is not structure that will repeat in the future. This issue is known as **in-sample over-fitting**. It is a recurrent question for all statistical and machine learning methods, and in credit it acquires a regulatory edge, since a PD model that has memorised its development sample will fail its out-of-time monitoring. It will be discussed in detail in the lectures that follow.

We are now ready to take off on this process of statistical modelling and machine learning applied to credit risk: please enjoy!

Copyright and attribution

References

- Bondora AS. *Loan dataset (public reports)*. https://www.bondora.com/en/public-reports
- Botha, A., Oberholzer, E., Larney, J. and de Jongh, R. (2023). *Defining and comparing SICR-events for classifying impaired loans under IFRS 9*.
- Botha, A. and Verster, T. (2025). *Approaches for modelling the term-structure of default risk under IFRS 9: a tutorial using discrete-time survival analysis*.
- Breeden, J.L. (2016). *Incorporating lifecycle and environment in loan-level forecasts and stress tests*. European Journal of Operational Research.
- International Accounting Standards Board (2014). *IFRS 9: Financial Instruments*, section 5.5: impairment.
- European Parliament and Council. *Regulation (EU) No 575/2013 (Capital Requirements Regulation)*, Article 178: default of an obligor, as amended by Regulation (EU) 2024/1623 (CRR3), and Article 180: requirements specific to PD estimation.
- European Banking Authority (2017). *Guidelines on PD estimation, LGD estimation and the treatment of defaulted exposures* (EBA/GL/2017/16).
- Vasicek, O.A. (2002). *The distribution of loan portfolio value*. Risk, 15(12).
- Prudential Regulation Authority (2024). *SS3/24: Credit risk, definition of default*.
- Wüthrich, M.V. and Merz, M. (2023). *Statistical Foundations of Actuarial Learning and its Applications*. Springer.
- Scognamiglio, S., Maggi, M. and Wüthrich, M.V. (2026). *Deep Learning for Actuarial Modeling*, SSRN 5162304.